

Uso de álgebras modernas para seguridad y criptografía

Tabla de contenidos

1	Teoría de anillos y campos aplicada a la criptografía	2
1.1	Definiciones y propiedades básicas	2
1.2	El algoritmo de Euclides	4
1.3	Aritmética y álgebra modular	8
1.4	Exponenciación binaria	9
2	Protocolos de encriptación clásicos	10
2.1	El sistema RSA	11
2.1.1	Protocolo RSA para la generación de claves	11
2.1.2	Protocolo RSA para el cifrado de mensajes	12
2.1.3	Protocolo RSA para el descifrado de mensajes	12
2.1.4	Protocolo RSA para la firma de mensajes	13
2.1.5	Protocolo RSA para la verificación de firmas	13
2.1.6	Ejemplo de aplicación del sistema RSA	14
2.1.7	Detalles técnicos del sistema RSA	15
2.2	El sistema de Diffie-Hellman	16
2.2.1	Protocolo para la generación de una clave secreta compartida	16
2.2.2	Ejemplo de aplicación del sistema Diffie-Hellman	17
2.3	El sistema de encriptación ElGamal	18
2.3.1	Protocolo ElGamal para la generación de claves	18
2.3.2	Protocolo ElGamal para el cifrado de mensajes	19
2.3.3	Protocolo ElGamal para el descifrado de mensajes	19
2.3.4	Ejemplo de aplicación del sistema ElGamal	20
2.3.5	Detalles técnicos del sistema ElGamal	20
3	Protocolos de encriptación de curvas elípticas	21
3.1	Curvas elípticas	21
3.2	El sistema Diffie-Hellman de curvas elípticas (ECDH)	23
3.2.1	Protocolo para la generación de una clave secreta compartida	23

3.2.2	Ejemplo de aplicación del sistema ECDH	24
3.3	El sistema para la generación de firmas digitales de curvas elípticas (ECDSA)	25
3.3.1	Protocolo ECDSA para la generación de claves	25
3.3.2	Protocolo ECDSA para la generación de firmas digitales	25
3.3.3	Protocolo ECDSA para la verificación de firmas digitales	26
3.3.4	Ejemplo de aplicación del sistema ECDSA	26
3.3.5	Detalles técnicos del sistema ECDSA	27

1. Teoría de anillos y campos aplicada a la criptografía

En esta sección presentaremos los resultados básicos de la teoría de anillos y campos que serán necesarios para el desarrollo de los temas relacionados con la criptografía.

1.1. Definiciones y propiedades básicas

Definición 1.1. Un *anillo* es un conjunto A junto con dos operaciones binarias,

$$\begin{aligned} \text{Suma} : A \times A &\longrightarrow A \\ (a_1, a_2) &\mapsto a_1 + a_2 \\ \text{Mult} : A \times A &\longrightarrow A \\ (a_1, a_2) &\mapsto a_1 \cdot a_2 \end{aligned}$$

junto con una operación unaria,

$$\begin{aligned} \text{Inv} : A &\longrightarrow A \\ a &\mapsto -a \end{aligned}$$

y elementos especiales $0, 1 \in A$ que satisfacen las siguientes propiedades:

1. **Asociatividad de la suma:** Dados $a_1, a_2, a_3 \in A$, se cumple que

$$(a_1 + a_2) + a_3 = a_1 + (a_2 + a_3).$$

2. **Elemento neutro de la suma:** Dado $a \in A$, se cumple que

$$a + 0 = 0 + a = a.$$

3. **Inverso aditivo:** Dado $a \in A$, se cumple que

$$a + (-a) = -a + a = 0.$$

4. **Conmutatividad de la suma:** Dados $a_1, a_2 \in A$, se cumple que

$$a_1 + a_2 = a_2 + a_1.$$

5. **Asociatividad de la multiplicación:** Dados $a_1, a_2, a_3 \in A$, se cumple que

$$(a_1 \cdot a_2) \cdot a_3 = a_1 \cdot (a_2 \cdot a_3).$$

6. **Elemento neutro de la multiplicación:** Dado $a \in A$, se cumple que

$$a \cdot 1 = 1 \cdot a = a.$$

7. **Distributividad:** Dados $a_1, a_2, a_3 \in A$, se cumple que

$$a_1 \cdot (a_2 + a_3) = a_1 \cdot a_2 + a_1 \cdot a_3 \quad \text{y} \quad (a_1 + a_2) \cdot a_3 = a_1 \cdot a_3 + a_2 \cdot a_3.$$

Si además tenemos que

8. **Conmutatividad de la multiplicación:** Dados $a_1, a_2 \in A$, se cumple que

$$a_1 \cdot a_2 = a_2 \cdot a_1,$$

entonces decimos que A es un *anillo conmutativo*. Si además se cumple que

9. **Inversos multiplicativos:** Dado $a \in A$, si $a \neq 0$, entonces existe $a^{-1} \in A$ tal que

$$a \cdot a^{-1} = a^{-1} \cdot a = 1,$$

entonces decimos que A es un *campo*.

Ejemplo 1.2. Algunos ejemplos de anillos y campos son los siguientes:

1. El conjunto de los enteros $\mathbb{Z}$ con las operaciones usuales de suma y multiplicación es un anillo conmutativo, pero no es un campo.
2. El conjunto de los números racionales $\mathbb{Q}$ con las operaciones usuales de suma y multiplicación es un campo.
3. El conjunto de los números reales $\mathbb{R}$ con las operaciones usuales de suma y multiplicación es un campo.
4. El conjunto de los números complejos $\mathbb{C}$ con las operaciones usuales de suma y multiplicación es un campo.
5. El conjunto de las matrices de 2×2 con entradas en $\mathbb{R}$ con las operaciones usuales de suma y multiplicación es un anillo que no es conmutativo.
6. Para todo entero $n > 0$ el conjunto $\mathbb{Z}/n\mathbb{Z}$ con las operaciones de suma y multiplicación definidas anteriormente es un anillo conmutativo.

Observación 1.3. La ventaja de tener una definición abstracta de anillo y campo es que, una vez que hemos demostrado un resultado para anillos o campos, este resultado se puede aplicar a todos los ejemplos anteriores sin necesidad de volver a demostrarlo para cada caso particular. Por ejemplo, consideremos el siguiente resultado:

Teorema 1.4. *Sea A un anillo. Entonces, para todo $a \in A$, se cumple que $a \cdot 0 = 0 \cdot a = 0$.*

Demostración. Para demostrar este resultado, notamos que para todo $a \in A$,

$$\begin{aligned} a \cdot (0 + 0) &= a \cdot 0 \\ a \cdot 0 + a \cdot 0 &= a \cdot 0 \\ (a \cdot 0 + a \cdot 0) + (-a \cdot 0) &= a \cdot 0 + (-a \cdot 0) \\ a \cdot 0 + (a \cdot 0 + (-a \cdot 0)) &= 0 \\ a \cdot 0 + 0 &= 0 \\ a \cdot 0 &= 0. \end{aligned}$$

De manera similar, se demuestra que $0 \cdot a = 0$. □

1.2. El algoritmo de Euclides

Discutiremos ahora el algoritmo de Euclides, el cual será una herramienta necesaria para el cálculo de inversos multiplicativos en aritmética modular. Pero antes de presentar el algoritmo, necesitaremos introducir algunas definiciones y resultados preliminares.

Definición 1.5. Supongamos que tenemos dos números enteros $n > m \geq 0$. Decimos que d es un *máximo común divisor* de n y m si se satisfacen las siguientes condiciones:

1. $d > 0$.
2. $d|n$ y $d|m$.
3. Si $t|n$ y $t|m$, entonces $t|d$.

Ejemplo 1.6. Consideremos los siguientes ejemplos:

1. Supongamos que

$$n = 6 = 2 \cdot 3, \quad m = 4 = 2^2.$$

En este caso, si tomamos $d = 2$, vemos que d es un máximo común divisor de 6 y 4.

2. Si

$$n = 36 = 2^2 \cdot 3^2, \quad m = 24 = 2^3 \cdot 3,$$

entonces

$$d = 2^2 \cdot 3 = 12$$

es un máximo común divisor de 36 y 24.

Lemma 1.7. *Supongamos que $t|n$ y $t|m$. Entonces, para todo a y $b \in \mathbb{Z}$, se cumple que $t|an + bm$.*

Demostración. Para demostrar este resultado, notemos que, como $t|n$, existe $k \in \mathbb{Z}$ tal que $n = tk$. De manera similar, como $t|m$, existe $l \in \mathbb{Z}$ tal que $m = tl$. Por lo tanto,

$$\begin{aligned} an + bm &= a(tk) + b(tl) \\ &= t(ak + bl) \end{aligned}$$

de donde concluimos que $t|an + bm$. □

Ya con estos resultados a la mano, introduciremos el algoritmo de Euclides extendido y sus propiedades fundamentales.

Teorema 1.8 (Algoritmo de Euclides extendido). *Supongamos que $n > m \geq 0$. Entonces:*

1. *Existe $d \in \mathbb{Z}$ tal que d es un máximo común divisor de n y m .*
2. *Existen $a, b \in \mathbb{Z}$ tales que*

$$an + bm = d.$$

Demostración. Para la demostración de este resultado, usaremos la interpretación BHK (Brouwer-Heyting-Kolmogorov) que nos indica que la demostración de un teorema de existencia debe tener la siguiente estructura:

1. Debemos presentar un *algoritmo* que nos permita calcular los número d , a y b a partir del par de enteros $n > m \geq 0$.
2. Debemos demostrar que los números d , a y b obtenidos en el algoritmo anterior satisfacen las propiedades requeridas.

Empezaremos por indicar un algoritmo para calcular los números d , a y b a partir de $n > m \geq 0$. Para esto definiremos una función recursiva

$$(a, b, d) = \text{emcd}(n, m)$$

de la siguiente manera:

1. Si $m = 0$, definimos

$$(a, b, d) = \text{emcd}(n, m) = (1, 0, n).$$

2. Si $m \neq 0$, entonces, usando el algoritmo de la división, encontramos q, r , con $0 \leq r < m$ tales que

$$n = qm + r.$$

En este caso, definimos

$$(a, b, d) = (\text{emcd}(m, r)_2, \text{emcd}(m, r)_1 - q \text{emcd}(m, r)_2, \text{emcd}(m, r)_3),$$

donde $\text{emcd}(m, r)_1$, $\text{emcd}(m, r)_2$ y $\text{emcd}(m, r)_3$ representan la primera, segunda y tercera entrada, respectivamente, de la tripleta asociada a $\text{emcd}(m, r)$.

Antes de continuar con la demostración, ilustraremos el funcionamiento del algoritmo mediante el análisis de algunos casos particulares:

1. Cuando $n = 6$, $m = 4$, obtenemos:

$$\begin{aligned} \text{emcd}(6, 4) &= (1, 0 - (1)(1), 2) = (1, -1, 2) & q_1 &= 1, & r_1 &= 2 \\ \text{emcd}(4, 2) &= (0, 1 - (2)(0), 2) = (0, 1, 2) & q_2 &= 2, & r_2 &= 0 \\ \text{emcd}(2, 0) &= (1, 0, 2) \end{aligned}$$

2. Cuando $n = 32$, $m = 24$, obtenemos:

$$\begin{aligned} \text{emcd}(32, 24) &= (1, 0 - (1)(1), 8) = (1, -1, 8) & q_1 &= 1, & r_1 &= 8 \\ \text{emcd}(24, 8) &= (0, 1 - (3)(0), 8) = (0, 1, 8) & q_2 &= 3, & r_2 &= 0 \\ \text{emcd}(8, 0) &= (1, 0, 8) \end{aligned}$$

3. Cuando $n = 37$, $m = 14$ obtenemos:

$$\begin{aligned} \text{emcd}(37, 14) &= (-3, 2 - (2)(-3), 1) = (-3, 8, 1) & q_1 &= 2, & r_1 &= 9 \\ \text{emcd}(14, 9) &= (2, -1 - (1)(2), 1) = (2, -3, 1) & q_2 &= 1, & r_2 &= 5 \\ \text{emcd}(9, 5) &= (-1, 1 - (1)(-1), 1) = (-1, 2, 1) & q_3 &= 1, & r_3 &= 4 \\ \text{emcd}(5, 4) &= (1, 0 - (1)(1), 1) = (1, -1, 1) & q_4 &= 1, & r_4 &= 1 \\ \text{emcd}(4, 1) &= (0, 1 - (4)(0), 1) = (0, 1, 1) & q_5 &= 4, & r_5 &= 0 \\ \text{emcd}(1, 0) &= (1, 0, 1) \end{aligned}$$

Regresemos ahora a la demostración del teorema. Hasta ahora hemos proporcionado un algoritmo para calcular los valores

$$(a, b, d) = \text{emcd}(n, m).$$

De acuerdo con la interpretación BHK, ahora debemos demostrar que estos números satisfacen las condiciones requeridas. Empezaremos por notar que la estructura general del algoritmo es la siguiente:

$$\begin{array}{lll} \text{emcd}(n, m) & n = q_1 m + r_1, & r_1 = n - q_1 m. \\ \text{emcd}(m, r_1) & m = q_2 r_1 + r_2, & r_2 = m - q_2 r_1 \\ \text{emcd}(r_1, r_2) & r_1 = q_3 r_2 + r_3, & r_3 = r_1 - q_3 r_2 \\ \vdots & \vdots & \vdots \\ \text{emcd}(r_{j-1}, r_j) & r_{j-1} = q_{j+1} r_j + r_{j+1}, & r_{j+1} = r_{j-1} - q_{j+1} r_j \\ \text{emcd}(r_j, r_{j+1}) & r_j = q_{j+2} r_{j+1} + r_{j+2}, & r_{j+2} = r_j - q_{j+2} r_{j+1} \\ \vdots & \vdots & \vdots \\ \text{emcd}(r_{k-1}, r_k) & r_{k-1} = q_{k+1} r_k + r_{k+1}, & r_{k+1} = r_{k-1} - q_{k+1} r_k. \\ \text{emcd}(r_k, r_{k+1}) & r_k = q_{k+2} r_{k+1}, & 0 = r_k - q_{k+2} r_{k+1}. \\ \text{emcd}(r_{k+1}, 0) & & \end{array}$$

Notemos además que, de acuerdo al algoritmo que describimos al principio de la demostración, el valor de d es precisamente r_{k+1} . Por lo tanto, para demostrar que d es un máximo común divisor de n y m , debemos demostrar que r_{k+1} satisface las propiedades requeridas:

1. **$d > 0$.** Notemos que, por la construcción del algoritmo, $r_j \geq 0$ para todo $j \in \mathbb{Z}$. Por otro lado, como r_{k+1} es el último residuo no nulo del algoritmo, se sigue que $d = r_{k+1} > 0$.
2. **$d|n$ y $d|m$.** Para demostrar esta propiedad, procederemos por inducción.

Caso base. Empezaremos por demostrar que $d|r_k$ y $d|r_{k+1}$. Para esto, notemos primero que como $d = r_{k+1} > 0$, entonces $d|r_{k+1}$ trivialmente. Por otro lado, como $r_k = q_{k+2}r_{k+1}$, se sigue que $d|r_k$.

Paso inductivo. Supongamos que $d|r_j$ y $d|r_{j+1}$ para algún $j \in \mathbb{Z}$ con $0 \leq j < k$. Queremos demostrar que, en este caso, $d|r_{j-1}$. Para esto notamos que, como $r_{j-1} = q_{j+1}r_j + r_{j+1}$, entonces, del lema 1.7, obtenemos que $d|r_{j-1}$.

3. **Si $t|n$ y $t|m$, entonces $t|d$.** Esta propiedad también la demostraremos por inducción.

Caso base. Por hipótesis, $t|n$ y $t|m$.

Paso inductivo. Supongamos que $t|r_{j-1}$ y $t|r_j$ para algún $j \in \mathbb{Z}$ con $0 \leq j < k$. Queremos demostrar que, en este caso, $t|r_{j+1}$. Para esto notamos que, como $r_{j+1} = r_{j-1} - q_{j+1}r_j$, entonces, del lema 1.7, obtenemos que $t|r_{j+1}$.

Lo único que nos queda por demostrar es que si $(a, b, d) = \text{emcd}(n, m)$ entonces la identidad $d = an + bm$ se satisface. Para demostrar esta identidad también usaremos inducción:

Caso base. Notamos que

$$\text{emcd}(r_{k+1}, 0) = (1, 0, r_{k+1}),$$

y, claramente,

$$d = r_{k+1} = r_{k+1} \cdot 1 + 0 \cdot 0.$$

Paso Inductivo. Supongamos que

$$\text{emcd}(r_j, r_{j+1}) = (a_j, b_j, r_{k+1}),$$

satisface las propiedades requeridas, es decir, supongamos que

$$a_j r_j + b_j r_{j+1} = r_{k+1}. \tag{1}$$

Queremos demostrar que

$$\text{emcd}(r_{j-1}, r_j) = (a_{j-1}, b_{j-1}, r_{k+1}) = (b_j, a_j - q_{j+1}b_j, r_{k+1}),$$

en otras palabras, queremos demostrar que

$$r_{k+1} = b_j r_{j-1} + (a_j - q_{j+1}b_j) r_j.$$

Pero, como $r_{j-1} = q_{j+1}r_j + r_{j+1}$, tenemos que

$$\begin{aligned} b_j r_{j-1} + (a_j - q_{j+1}b_j)r_j &= b_j(q_{j+1}r_j + r_{j+1}) + (a_j - q_{j+1}b_j)r_j \\ &= b_j q_{j+1}r_j + b_j r_{j+1} + a_j r_j - q_{j+1}b_j r_j \\ &= a_j r_j + b_j r_{j+1} \\ &= r_{j+1}, \end{aligned}$$

donde en el último paso hemos usado la ecuación (1).

□

1.3. Aritmética y álgebra modular

Como habíamos mencionado anteriormente, el algoritmo de Euclides extendido puede ser utilizado para calcular inversos multiplicativos en aritmética modular. De manera más explícita, supongamos que tenemos enteros $n > m \geq 0$ y supongamos que $\text{mcd}(n, m) = 1$. En este caso, el algoritmo de Euclides extendido nos proporciona enteros a y b tales que

$$an + bm = 1.$$

Se sigue que

$$\begin{aligned} an + bm &= 1 \pmod{n} \\ bm &= 1 \pmod{n}. \end{aligned}$$

En otras palabras, b es un inverso multiplicativo de m en la aritmética módulo n . En particular, si n es un número primo, entonces para todo $m \in \mathbb{Z}$ con $0 < m < n$, se cumple que $\text{mcd}(n, m) = 1$, por lo que el algoritmo de Euclides extendido nos permite calcular un inverso multiplicativo de m módulo n . Se sigue que, cuando n es un número primo, el conjunto $\mathbb{Z}/n\mathbb{Z}$ es un campo.

Ejemplo 1.9. Encuentre el valor de x tal que

$$3x + 7 = 0 \pmod{11}.$$

Solución. Para resolver esta ecuación, empezamos por notar que

$$\begin{aligned} 3x + 7 &= 0 \pmod{11} \\ 3x &= -7 \pmod{11} \\ 3x &= 4 \pmod{11}. \end{aligned}$$

Para continuar, notamos que $\text{mcd}(11, 3) = 1$, por lo que podemos utilizar el algoritmo de Euclides extendido para encontrar un inverso multiplicativo de 3 módulo 11. Para esto, calculamos

$$\begin{array}{lll} \text{emcd}(11, 3) = (-1, 1 - (-1)3, 1) = (-1, 4, 1) & q_1 = 3, & r_1 = 2 \\ \text{emcd}(3, 2) = (1, 0 - (1)(1), 1) = (1, -1, 1) & q_2 = 1, & r_2 = 1 \\ \text{emcd}(2, 1) = (0, 1 - (2)(0), 1) = (0, 1, 1) & q_3 = 2, & r_3 = 0 \\ \text{emcd}(1, 0) = (1, 0, 1) & & \end{array}$$

Se sigue que 4 es el inverso multiplicativo de 3 módulo 11. Por lo tanto, multiplicamos ambos lados de la ecuación por este número para obtener

$$\begin{aligned} 3x &= 4 \pmod{11} \\ 4 \cdot 3x &= 4 \cdot 4 \pmod{11} \\ 12x &= 16 \pmod{11} \\ x &= 5 \pmod{11}. \end{aligned}$$

1.4. Exponenciación binaria

Supongamos que queremos calcular

$$a^k \pmod{n}$$

para algunos enteros a , k y n . Para esto, podemos usar el algoritmo de exponenciación binaria, el cual se basa en la siguiente idea: si k es un número par, entonces

$$a^k = (a^2)^{\frac{k}{2}}.$$

Si k es un número impar, entonces

$$a^k = a \cdot a^{k-1} = a \cdot (a^2)^{\frac{k-1}{2}}.$$

Este algoritmo es mucho más eficiente que el método tradicional de calcular a^k multiplicando a por sí mismo k veces, ya que el número de multiplicaciones necesarias para calcular a^k usando el método tradicional es $k - 1$, mientras que el número de multiplicaciones necesarias para calcular a^k usando el algoritmo de exponenciación binaria es aproximadamente $\log_2(k)$.

Ejemplo 1.10. Para calcular

$$3^{19} \pmod{23},$$

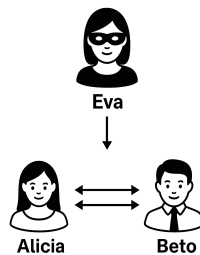
realizamos las siguientes operaciones:

$$\begin{aligned}
 3^{19} &= 3 \cdot 3^{18} \quad \text{mód } 23 \\
 &= 3 \cdot (3^2)^9 \quad \text{mód } 23 \\
 &= 3 \cdot 9^9 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot 9^8 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot (9^2)^4 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot (81)^4 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot 12^4 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot (12^2)^2 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot (144)^2 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot 6^2 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot 36 \quad \text{mód } 23 \\
 &= 3 \cdot 9 \cdot 13 \quad \text{mód } 23 \\
 &= 3 \cdot 117 \quad \text{mód } 23 \\
 &= 3 \cdot 2 \quad \text{mód } 23 \\
 &= 6 \quad \text{mód } 23.
 \end{aligned}$$

Notamos que este procedimiento es mucho más rápido que multiplicar 3 por sí mismo 19 veces.

2. Protocolos de encriptación clásicos

En esta sección consideraremos los protocolos criptográficos clásicos, es decir, aquellos basados en aritmética modular y que no utilizan curvas elípticas. El contexto estándar de estos protocolos es el siguiente: supongamos que tenemos dos agentes, digamos Alicia y Beto, que desean intercambiar información a través de un canal inseguro. Supongamos además que tenemos un agente hostil, digamos Eva, que puede interceptar toda la comunicación que se envía a través de este canal.



El objetivo de nuestros protocolos es asegurarnos de que Alicia y Beto puedan comunicarse de manera segura, es decir, que Eva no pueda leer la información que se están transmitiendo. Dependiendo del

tipo de información que se desea intercambiar, podemos utilizar diferentes protocolos criptográficos que nos permitan lograr este objetivo.

2.1. El sistema RSA

El sistema RSA, introducido por Rivest, Shamir y Adleman en 1977, es un sistema criptográfico de clave pública que se utiliza ampliamente en la industria. Este sistema consiste de cinco algoritmos:

1. Un algoritmo para la generación de una *clave pública* y una *clave privada*.
2. Un algoritmo para el *cifrado de mensajes*.
3. Un algoritmo para el *descifrado de mensajes*.
4. Un algoritmo para la *firma de mensajes*.
5. Un algoritmo para la *verificación de firmas*.

2.1.1. Protocolo RSA para la generación de claves

El algoritmo para la generación de las claves pública y privada consiste de los siguientes pasos:

1. Alicia selecciona dos números primos aleatorios p y q .
2. Alicia calcula

$$n = pq, \quad \phi(n) = (p - 1)(q - 1).$$

3. Alicia elige un número $0 < e < \phi(n)$ tal que

$$\text{mcd}(\phi(n), e) = 1.$$

4. Alicia calcula un $1 < d < \phi(n)$ tal que

$$ed = 1 \quad \text{mód } \phi(n).$$

5. Alicia define su clave pública y privada como

$$\begin{aligned} \text{clave_publica} &= (e, n) \\ \text{clave_privada} &= (d, n) \end{aligned}$$

y comunica su clave pública a Beto.

Notas: Con respecto al algoritmo para la generación de claves, notemos lo siguiente:

1. Eva también puede conocer la clave pública de Alicia, pero no su clave privada.

2. En la práctica, el número e es comúnmente $e = 3$ o $e = 65537$. Estos números son elegidos por razones de eficiencia, ya que permiten realizar el cifrado de manera más rápida. Sin embargo, es importante asegurarse de que $\text{mcd}(\phi(n), e) = 1$.
3. En la práctica, el número d es calculado utilizando el algoritmo de Euclides extendido, el cual permite encontrar el inverso multiplicativo de e módulo $\phi(n)$. Este algoritmo es eficiente y se puede implementar fácilmente en cualquier lenguaje de programación.
4. En la práctica, el número d es comúnmente almacenado en un formato binario, lo cual permite una representación más compacta y eficiente. Sin embargo, es importante asegurarse de que el número d sea almacenado de manera segura y no sea accesible a agentes no autorizados.
5. Notemos que si logramos factorizar n en sus factores primos p y q , entonces podemos calcular $\phi(n)$ y, por lo tanto, podemos calcular la clave privada d . Por esta razón, la seguridad del sistema RSA depende de la dificultad de factorizar números enteros grandes. En general, se recomienda utilizar números primos de al menos 1024 bits para asegurar la integridad del sistema.

2.1.2. Protocolo RSA para el cifrado de mensajes

El algoritmo consiste de los siguientes pasos:

1. Beto selecciona un mensaje m_b tal que $0 < m_b < n$ y define

$$\text{mensaje_llano} = (m_b, n)$$

2. Beto calcula

$$c = m_b^e \quad \text{mód } n.$$

y define

$$\text{mensaje_cifrado} = (c, n).$$

Este es el mensaje cifrado que Beto puede compartir con Alicia a través del canal inseguro.

2.1.3. Protocolo RSA para el descifrado de mensajes

El algoritmo consiste de los siguientes pasos:

1. Alicia calcula

$$\tilde{m}_b = c^d \quad \text{mód } n$$

y define

$$\text{mensaje_descifrado} = (\tilde{m}_b, n).$$

2. Alicia puede proceder ahora a leer el contenido $\tilde{m}_b$ del mensaje descifrado.

Notas:

1. Notemos que como Eva no conoce la clave privada de Alicia, no puede descifrar el mensaje cifrado c . Por lo tanto, el mensaje cifrado es seguro y no puede ser leído por Eva.
2. En la práctica, el sistema de encriptación RSA es mucho más ineficiente que el uso de algoritmos de cifrado simétrico como AES. Como veremos en la siguiente sección, el algoritmo de Diffie-Hellman permite acordar una clave secreta entre dos agentes, la cual puede ser utilizada posteriormente para cifrar la comunicación entre ellos. Es por eso que este algoritmo ya no es tan utilizado en la práctica. Sin embargo, el algoritmo RSA sigue siendo ampliamente utilizado para la firma digital de mensajes y la verificación de firmas.

2.1.4. Protocolo RSA para la firma de mensajes

El algoritmo consiste de los siguientes pasos:

1. Alicia selecciona un mensaje m_a tal que $0 < m_a < n$ y define

$$\text{mensaje_llano} = (m_a, n).$$

2. Alicia utiliza una función de hash para calcular el hash del mensaje m_a , es decir, calcula

$$h = \text{Hash}(m_a).$$

3. Alicia calcula

$$s = h^d \quad \text{mód } n$$

y define

$$\text{mensaje_firmado} = (m_a, s).$$

Este es el mensaje firmado que Alicia puede compartir con Beto a través del canal inseguro.

2.1.5. Protocolo RSA para la verificación de firmas

El algoritmo consiste de los siguientes pasos:

1. Beto utiliza una función de hash para calcular el hash del mensaje m_a , es decir, calcula

$$h = \text{Hash}(m_a).$$

2. Beto calcula

$$\tilde{h} = s^e \quad \text{mód } n$$

y verifica que

$$h = \tilde{h} \quad \text{mód } n.$$

Notas: En la práctica, el hash h es comúnmente calculado utilizando una función de hasheo como SHA-256 o SHA-512, lo cual permite una representación más compacta y eficiente. Sin embargo, es importante asegurarse de que la función de hasheo utilizada sea segura y no sea vulnerable a ataques de colisión.

En general los algoritmos de generación de firma digital y su verificación buscan cumplir con los siguientes objetivos:

1. **Autenticidad:** El receptor del mensaje debe poder verificar que el mensaje fue realmente enviado por el remitente y no por un agente malicioso. Esto se logra utilizando la clave privada del remitente para firmar el mensaje y la clave pública del remitente para verificar la firma. Como solamente Alicia tiene acceso a su clave privada, solamente ella puede firmar el mensaje.
2. **Integridad:** El receptor del mensaje debe poder verificar que el mensaje no ha sido modificado durante la transmisión. Esto se logra utilizando una función de hasheo para calcular el hash del mensaje y compararlo con el hash firmado por el remitente. Si los hashes son iguales, entonces el mensaje no ha sido modificado.
3. **No repudio:** El remitente del mensaje no puede negar haber enviado el mensaje. Esto se logra utilizando la clave privada del remitente para firmar el mensaje y la clave pública del remitente para verificar la firma. Como solamente Alicia tiene acceso a su clave privada, solamente ella puede firmar el mensaje y por tanto no puede negar haber enviado el mensaje ni repudiar el contenido del mismo.

2.1.6. Ejemplo de aplicación del sistema RSA

Ilustraremos ahora los algoritmos del protocolo RSA con ejemplos concretos.

1. **Generación de claves.** Tomaremos

$$p = 5, \quad q = 11.$$

Entonces tendremos que

$$n = 5(11) = 55, \quad \phi(n) = 4(10) = 40.$$

Elegimos ahora $e = 3$ y observamos que

$$\text{mcd}(40, 3) = 1.$$

Utilizando el algoritmo de Euclides extendido, calculamos el inverso de 3 módulo 40 para obtener

$$d = 27.$$

Por lo tanto, tenemos que

$$\begin{aligned} \text{clave_publica} &= (3, 55) \\ \text{clave_privada} &= (27, 55). \end{aligned}$$

2. **Cifrado de mensajes.** Supongamos que Beto quiere enviar el mensaje $m_b = 7$ a Alicia. Entonces, Beto calcula

$$c = 7^3 = 13 \quad \text{mód } 55.$$

Por lo tanto, el mensaje cifrado que Beto envía a Alicia es

$$\text{mensaje_cifrado} = (13, 55).$$

3. **Descifrado de mensajes.** Alicia recibe el mensaje cifrado $c = 13$ y calcula

$$\tilde{m}_b = 13^{27} = 7 \quad \text{mód } 55.$$

Notemos que el mensaje descifrado es el mismo que el mensaje original enviado por Beto. Por lo tanto, Alicia puede leer el mensaje descifrado $\tilde{m}_b = 7$.

4. **Firma de mensajes.** Supongamos que Alicia quiere firmar el mensaje $m_a = 9$. Entonces, Alicia define

$$\text{mensaje_llano} = (9, 55).$$

Para este ejemplo, consideraremos la función de hash como la función identidad, es decir,

$$h = \text{Hash}(m_a) = m_a = 9.$$

Por tanto Alicia debe de calcular

$$s = 9^{27} = 4 \quad \text{mód } 55.$$

Se sigue que el mensaje firmado que Alicia envía a Beto es

$$\text{mensaje_firmado} = (9, 4).$$

5. **Verificación de firmas.** Beto recibe el mensaje firmado $(9, 4)$ y calcula

$$\tilde{h} = 4^3 = 9 \quad \text{mód } 55.$$

Beto verifica que el mensaje firmado es correcto al comprobar que

$$\tilde{h} = 9 = h \quad \text{mód } 55.$$

2.1.7. Detalles técnicos del sistema RSA

Para aplicar y entender el sistema RSA, debemos resolver los siguientes problemas técnicos:

1. En el paso 4 del algoritmo RSA para la generación de claves debemos resolver la ecuación

$$ed = 1 \quad \text{mód } \phi(n) \tag{2}$$

bajo la condición de que $\text{mcd}(\phi(n), e) = 1$. El algoritmo de Euclides extendido nos proporciona un método eficiente para calcular d .

2. Demostrar que, efectivamente,

$$\tilde{m} = c^d = (m^e)^d = m^{ed} = m \quad \text{mód } n.$$

Para esto notemos que de la ecuación (2) se sigue que $ed = 1 + k\phi(n)$ para algún entero k . Por lo tanto,

$$\begin{aligned}\tilde{m} &= m^{ed} \quad \text{mód } n \\ &= m^{1+k\phi(n)} \quad \text{mód } n \\ &= m(m^{\phi(n)})^k \quad \text{mód } n.\end{aligned}$$

Ahora, por el teorema de Lagrange, tenemos que $m^{\phi(n)} = 1 \quad \text{mód } n$. De aquí se sigue que

$$\tilde{m} = m \quad \text{mód } n,$$

tal y como queríamos demostrar.

3. Estimar la seguridad del sistema. En particular, tratar de estimar el tiempo que tomaría romper el sistema cuando se desconoce la clave privada.

Observación 2.1. En la práctica, los valores de p y q deben ser muy grandes para asegurar la integridad del sistema. Por ejemplo, si tomamos $p = 5$, $q = 3$ y $e = 3$, entonces la siguiente tabla nos da la información necesaria para romper este protocolo:

m	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
c	0	1	8	12	4	5	6	13	2	9	10	11	3	7	14

2.2. El sistema de Diffie-Hellman

El objetivo del sistema de Diffie-Hellman es que Alicia y Beto puedan acordar una clave secreta, la cual podrá ser utilizada posteriormente para cifrar la comunicación entre ellos utilizando un algoritmo de cifrado simétrico como AES.

2.2.1. Protocolo para la generación de una clave secreta compartida

Los pasos del algoritmo son los siguientes:

1. Alicia y Beto acuerdan un número primo p y una base g . Estos números pueden ser públicos y conocidos por todos los agentes, incluyendo Eva.
2. Alicia y Beto eligen claves privadas a y b respectivamente, las cuales son números enteros aleatorios menores que p . Estas claves son secretas y no deben ser compartidas con nadie.

3. Alicia y Beto calculan sus claves públicas A y B respectivamente, utilizando la siguiente fórmula:

$$A = g^a \pmod{p}$$

para la clave pública de Alicia y

$$B = g^b \pmod{p}$$

para la clave pública de Beto. Estas claves públicas son enviadas a través del canal inseguro y pueden ser interceptadas por Eva.

4. Alicia y Beto calculan la clave secreta compartida s utilizando la siguiente fórmula:

$$s_A = B^a = (g^b)^a = g^{ba} \pmod{p}$$

para Alicia y

$$s_B = A^b = (g^a)^b = g^{ab} \pmod{p}$$

para Beto. Notamos que $s_A = s_B$, es decir, ambos agentes han acordado la misma clave secreta.

Nota: Para elegir los parámetros p y g es común utilizar una de las opciones listadas en la especificación [RFC 3526](#) publicada por el IETF en el 2003. Esta especificación incluye una lista de números primos de 1536, 2048, 3072, 4096, 6144 y 8192 bits, así como sus respectivas bases.

En general, un *primo de Sophie Germain* es un número primo q tal que $p = 2q + 1$ también es primo. En este caso, decimos que p es un *primo seguro*. Todos los primos de la lista de la especificación RFC 3526 son primos seguros. En este caso, para evitar ataques de subgrupos pequeños, se recomienda que las claves públicas A y B pertenezcan al subgrupo de orden g generado por g^2 . Para esto basta generar números aleatorios $\tilde{a}, \tilde{b} \leq q - 1$ y tomar $a = 2\tilde{a}$ y $b = 2\tilde{b}$. En general, para asegurar que el protocolo es resistente a ataques por fuerza bruta, se recomienda que $\tilde{a}$ y $\tilde{b}$ sean números aleatorios de al menos 256 bits. Notemos que la seguridad de este protocolo depende de la dificultad de resolver el problema del logaritmo discreto, es decir, dado g, p y $A = g^a \pmod{p}$, encontrar el número a . Como vimos anteriormente, el cálculo de $g^a \pmod{p}$ se puede realizar de manera eficiente utilizando exponenciación binaria, pero el cálculo del logaritmo discreto es un problema difícil y no se conoce ningún algoritmo eficiente para resolverlo en general. Por esta razón, el protocolo de Diffie-Hellman es considerado seguro y resistente a ataques por fuerza bruta.

2.2.2. Ejemplo de aplicación del sistema Diffie-Hellman

Ilustraremos ahora el sistema Diffie-Hellman con un ejemplo concreto. Alicia y Beto acuerdan el número primo $p = 23$ y la base $g = 5$. A continuación, Alicia elige la clave privada $a = 6$ y Beto elige la clave privada $b = 15$. Para calcular las claves públicas, Alicia y Beto utilizan las fórmulas dadas anteriormente:

$$A = 5^6 = 8 \pmod{23}$$

$$B = 5^{15} = 19 \pmod{23}$$

Finalmente, Alicia y Beto calculan la clave secreta compartida.

$$\begin{aligned}s_A &= 19^6 = 2 \pmod{23} \\ s_B &= 8^{15} = 2 \pmod{23}\end{aligned}$$

Notamos que $s_A = s_B = 2$, es decir, ambos agentes han acordado la misma clave secreta.

2.3. El sistema de encriptación ElGamal

El sistema de encriptación ElGamal, introducido por Taher ElGamal en 1985, es un protocolo de encriptación de clave pública basado en el protocolo de Diffie-Hellman. Este sistema consiste de tres algoritmos:

1. Un algoritmo para la generación de una clave pública y una clave privada.
2. Un algoritmo para el cifrado de mensajes.
3. Un algoritmo para el descifrado de mensajes.

2.3.1. Protocolo ElGamal para la generación de claves

El algoritmo para la generación de las claves pública y privada consiste de los siguientes pasos:

1. Alicia selecciona un número primo p y un generador g del grupo multiplicativo $(\mathbb{Z}/p\mathbb{Z})^*$. Estos números pueden ser públicos y conocidos por todos los agentes, incluyendo Eva.
2. Alicia elige de manera aleatoria un número x tal que

$$1 < x < p - 1$$

3. Alicia calcula

$$y = g^x \pmod{p}$$

y define su clave pública como

$$\text{clave_publica} = (g, y, p).$$

y su clave privada como

$$\text{clave_privada} = (g, x, p).$$

2.3.2. Protocolo ElGamal para el cifrado de mensajes

El algoritmo para el cifrado de mensajes consiste de los siguientes pasos:

1. Beto selecciona un mensaje m tal que $0 < m < p$ y define

$$\text{mensaje_llano} = (m, p).$$

2. Beto elige de manera aleatoria un número k tal que

$$1 < k < p - 1$$

3. Beto calcula

$$\begin{aligned} c_1 &= g^k \quad \text{mód } p \\ c_2 &= my^k \quad \text{mód } p \end{aligned}$$

4. Beto envía el mensaje cifrado

$$\text{mensaje_cifrado} = ((c_1, c_2), p)$$

a través del canal inseguro.

2.3.3. Protocolo ElGamal para el descifrado de mensajes

El algoritmo para el descifrado de mensajes consiste de los siguientes pasos:

1. Alicia recibe el mensaje cifrado $((c_1, c_2), p)$ y calcula

$$s = c_1^x \quad \text{mód } p$$

2. Alicia calcula el inverso multiplicativo de s módulo p , es decir, calcula un número s^{-1} tal que

$$ss^{-1} = 1 \quad \text{mód } p.$$

3. Alicia calcula

$$\tilde{m} = c_2 s^{-1} \quad \text{mód } p.$$

4. Alicia define el mensaje descifrado como

$$\text{mensaje_descifrado} = (\tilde{m}, p).$$

2.3.4. Ejemplo de aplicación del sistema ElGamal

Ilustraremos ahora los algoritmos del protocolo ElGamal con un ejemplo concreto.

1. **Generación de claves.** Alicia selecciona el número primo $p = 23$ y el generador $g = 5$. Alicia elige de manera aleatoria el número $x = 6$ y calcula

$$\begin{aligned}y &= 5^6 = 8 \pmod{23} \\ \text{clave_publica} &= (5, 8, 23) \\ \text{clave_privada} &= (5, 6, 23).\end{aligned}$$

2. **Cifrado de mensajes.** Supongamos que Beto quiere enviar el mensaje $m = 7$ a Alicia. Beto elige de manera aleatoria el número $k = 15$ y calcula

$$\begin{aligned}c_1 &= 5^{15} = 19 \pmod{23} \\ c_2 &= 7 \cdot 8^{15} = 14 \pmod{23}.\end{aligned}$$

Por lo tanto, el mensaje cifrado que Beto envía a Alicia es

$$\text{mensaje_cifrado} = ((19, 14), 23).$$

3. **Descifrado de mensajes.** Alicia recibe el mensaje cifrado $((19, 14), 23)$ y calcula

$$\begin{aligned}s &= 19^6 = 2 \pmod{23} \\ s^{-1} &= 12 \pmod{23} \\ \tilde{m} &= 14 \cdot 12 = 7 \pmod{23}.\end{aligned}$$

Por lo tanto, el mensaje descifrado que Alicia obtiene es $\tilde{m} = 7$, el mismo mensaje que Beto envió originalmente.

2.3.5. Detalles técnicos del sistema ElGamal

Recordemos que

$$y = g^x \pmod{p}$$

y que el mensaje cifrado es $((c_1, c_2), p)$, donde

$$\begin{aligned}c_1 &= g^k \pmod{p} \\ c_2 &= my^k \pmod{p}.\end{aligned}$$

Para descifrar el mensaje, Alicia calcula

$$\begin{aligned}s &= c_1^x \pmod{p} \\ &= (g^k)^x \pmod{p} \\ &= (g^x)^k \pmod{p} \\ &= y^k \pmod{p}.\end{aligned}$$

Como

$$c_2 = my^k \pmod{p},$$

tenemos que

$$\begin{aligned}\tilde{m} &= c_2 s^{-1} \pmod{p} \\ &= my^k s^{-1} \pmod{p} \\ &= my^k (y^k)^{-1} \pmod{p} \\ &= m \pmod{p}.\end{aligned}$$

Esto garantiza que el mensaje descifrado $\tilde{m}$ es igual al mensaje original m enviado por Beto.

3. Protocolos de encriptación de curvas elípticas

3.1. Curvas elípticas

Definición 3.1. Una curva elíptica, en forma de Weierstrass, es una curva de la forma

$$y^2 = x^3 + c_1x + c_0$$

que satisface la condición de que el discriminante

$$\Delta = -16(4c_1^3 + 27c_0^2)$$

es distinto de cero.

Observación 3.2. La condición de que el discriminante sea distinto de cero es equivalente a que la curva no tenga puntos singulares. Esto garantiza que la curva tenga una estructura algebraica adecuada para ser utilizada en criptografía.

A los puntos de una curva elíptica, junto con un punto especial llamado el “punto en el infinito”, se les puede dar una estructura de grupo abeliano. Las operaciones en este grupo, para puntos regulares

$$P = (x_1, y_1) \quad \text{y} \quad Q = (x_2, y_2)$$

con $x_1 \neq x_2$, están dadas por las siguientes fórmulas:

1. $P + Q = (x_3, y_3)$ donde

$$\begin{aligned}\lambda &= \frac{y_2 - y_1}{x_2 - x_1} \\ x_3 &= \lambda^2 - x_1 - x_2 \\ y_3 &= \lambda(x_1 - x_3) - y_1\end{aligned}$$

2. $2P = (x_4, y_4)$ donde

$$\begin{aligned}\mu &= \frac{3x_1^2 + c_1}{2y_1} \\ x_4 &= \mu^2 - 2x_1 \\ y_4 &= \mu(x_1 - x_4) - y_1\end{aligned}$$

3. $-P = (x_1, -y_1)$

Observación 3.3. Notemos que estas fórmulas están definidas sobre cualquier campo, pero en criptografía se suelen usar curvas elípticas definidas sobre campos finitos. Recordemos que en este caso la operación de división se realiza multiplicando por el inverso multiplicativo, el cual se puede calcular utilizando el algoritmo de Euclides extendido.

Ejemplo 3.4. Consideremos la curva elíptica E definida sobre el campo $\mathbb{F}_7 = \mathbb{Z}/7\mathbb{Z}$ y dada por la ecuación

$$y^2 = x^3 + x + 1$$

Notemos que en este caso el discriminante está dado por

$$\Delta = -16(4(1)^3 + 27(1)^2) = -16(4 + 27) = -16 \cdot 31 = 1 \pmod{7}$$

lo que nos garantiza que la curva no tiene puntos singulares. Los puntos de esta curva son

$$E(\mathbb{F}_7) = \{\infty, (0, 1), (0, 6), (2, 2), (2, 5)\}$$

Tomemos ahora $P = (0, 1)$ y $Q = (2, 2)$. Para calcular $P + Q$, primero calculamos λ :

$$\lambda = \frac{2 - 1}{2 - 0} = \frac{1}{2} = 4 \pmod{7}$$

Luego, calculamos x_3 y y_3 :

$$\begin{aligned}x_3 &= 4^2 - 0 - 2 = 16 - 2 = 14 = 0 \pmod{7} \\ y_3 &= 4(0 - 0) - 1 = -1 = 6 \pmod{7}\end{aligned}$$

Por lo tanto, $P + Q = (0, 6)$. Para calcular $2P$, primero calculamos μ :

$$\mu = \frac{3(0)^2 + 1}{2(1)} = \frac{1}{2} = 4 \pmod{7}$$

Luego, calculamos x_4 y y_4 :

$$\begin{aligned}x_4 &= 4^2 - 2(0) = 16 = 2 \pmod{7} \\ y_4 &= 4(0 - 2) - 1 = -8 - 1 = -9 = 5 \pmod{7}\end{aligned}$$

Por lo tanto, $2P = (2, 5)$. Finalmente, para calcular $-P$, simplemente negamos la coordenada y de P :

$$-P = (0, -1) = (0, 6) \pmod{7}$$

3.2. El sistema Diffie-Hellman de curvas elípticas (ECDH)

El sistema Diffie-Hellman de curvas elípticas (ECDH) es un sistema análogo al sistema Diffie-Hellman clásico, pero utiliza la estructura de grupo de una curva elíptica en lugar del grupo multiplicativo de un campo finito. Al igual que en el sistema clásico, el objetivo de ECDH es permitir que dos partes, digamos Alicia y Beto, establezcan una clave secreta compartida a través de un canal inseguro, que posteriormente se puede utilizar para cifrar la comunicación entre ellos utilizando un algoritmo de cifrado simétrico como AES.

3.2.1. Protocolo para la generación de una clave secreta compartida

Los pasos del algoritmo ECDH son los siguientes:

1. Alicia y Beto acuerdan utilizar una curva elíptica E definida sobre un campo finito $\mathbb{F}_p$, y un punto base $G \in E(\mathbb{F}_p)$ de orden n . Tanto la curva como el punto base y su orden son públicos, y pueden ser conocidos por todos los agentes, incluyendo Eva.
2. Alicia y Beto eligen claves privadas a y b respectivamente, las cuales son números enteros aleatorios menores que n . Estas claves son secretas y no deben ser compartidas con nadie.
3. Alicia y Beto calculan sus claves públicas A y B respectivamente, utilizando la siguiente fórmula:

$$A = aG \in E(\mathbb{F}_p)$$

para la clave pública de Alicia, y

$$B = bG \in E(\mathbb{F}_p)$$

para la clave pública de Beto. Estas claves públicas son enviadas a través del canal inseguro y pueden ser interceptadas por Eva.

4. Alicia y Beto calculan la clave secreta compartida S utilizando la siguiente fórmula:

$$S_A = aB = a(bG) = (ab)G \in E(\mathbb{F}_p)$$

para Alicia, y

$$S_B = bA = b(aG) = (ab)G \in E(\mathbb{F}_p)$$

para Beto. Notamos que $S_A = S_B$, es decir, ambos agentes han acordado la misma clave secreta.

Observación 3.5. Aunque en este caso la clave compartida es un punto S sobre la curva elíptica, en la práctica se suele utilizar la primera coordenada de este punto como la clave secreta compartida. Para asegurarse de que esta coordenada tenga la longitud adecuada para ser utilizada como clave de cifrado, se suele aplicar una función hash a esta coordenada antes de utilizarla como clave de cifrado.

Notemos también que, tal y como en el caso del protocolo Diffie-Hellman clásico, la seguridad del protocolo ECDH se basa en la dificultad del problema del logaritmo discreto en curvas elípticas. Es

decir, dado un punto P y un múltiplo kP de este punto, es computacionalmente difícil encontrar el entero k . Esto garantiza que incluso si Eva intercepta las claves públicas A y B , no podrá calcular la clave secreta compartida S sin conocer las claves privadas a o b .

Nota: Para elegir la curva elíptica y el punto base, se suelen utilizar curvas estándar que han sido ampliamente estudiadas y que se consideran seguras. Estas curvas estándar son publicadas por organizaciones como el NIST (National Institute of Standards and Technology) y el SECG (Standards for Efficient Cryptography Group). Por ejemplo, Bitcoin utiliza la curva secp256k1, la cual es una curva elíptica definida sobre el campo finito $\mathbb{F}_p$ con $p = 2^{256} - 2^{32} - 977$ y dada por la ecuación

$$y^2 = x^3 + 7$$

Más detalles sobre esta curva se pueden encontrar en el documento [SEC 2: Recommended Elliptic Curve Domain Parameters](#) publicado por el SECG.

3.2.2. Ejemplo de aplicación del sistema ECDH

Ilustraremos ahora el sistema ECDH con un ejemplo concreto. Alicia y Beto acuerdan utilizar la curva elíptica E definida sobre el campo finito $\mathbb{F}_{23} = \mathbb{Z}/23\mathbb{Z}$ y dada por la ecuación

$$y^2 = x^3 + 3x + 2$$

Notemos que el discriminante de esta curva está dado por

$$\Delta = 17 \pmod{23}$$

lo que nos garantiza que la curva no tiene puntos singulares. El punto base elegido por Alicia y Beto es $G = (5, 2)$, el cual tiene orden $n = 28$. Alicia elige la clave privada $a = 6$ y Beto elige la clave privada $b = 15$. Para calcular las claves públicas, Alicia y Beto utilizan las fórmulas dadas anteriormente:

$$\begin{aligned} A &= 6G = 6(5, 2) = (4, 3) \in E(\mathbb{F}_{23}) \\ B &= 15G = 15(5, 2) = (12, 15) \in E(\mathbb{F}_{23}) \end{aligned}$$

Finalmente, Alicia y Beto calculan la clave secreta compartida

$$\begin{aligned} S_A &= 6B = 6(12, 15) = (4, 3) \in E(\mathbb{F}_{23}) \\ S_B &= 15A = 15(4, 3) = (4, 3) \in E(\mathbb{F}_{23}) \end{aligned}$$

Notamos que $S_A = S_B$, es decir, ambos agentes han acordado la misma clave secreta.

3.3. El sistema para la generación de firmas digitales de curvas elípticas (ECDSA)

Consideraremos ahora el sistema para la generación de firmas digitales de curvas elípticas (ECDSA), el cual es un sistema de firma digital que utiliza la estructura de grupo de una curva elíptica. Al igual que en el caso del sistema ECDH, ECDSA se basa en la dificultad del problema del logaritmo discreto en curvas elípticas para garantizar la seguridad de las firmas digitales generadas con este sistema. Cabe destacar que en aplicaciones modernas de criptografía, el uso de criptografía de curvas elípticas es cada vez más común debido a su eficiencia y seguridad, especialmente en dispositivos con recursos limitados como teléfonos móviles y dispositivos IoT. ECDSA es ampliamente utilizado en protocolos de seguridad como TLS (Transport Layer Security) y en criptomonedas como Bitcoin para garantizar la autenticidad e integridad de las transacciones.

3.3.1. Protocolo ECDSA para la generación de claves

El algoritmo ECDSA para la generación de claves es el siguiente:

1. Alicia y Beto acuerda utilizar una curva elíptica E definida sobre un campo finito $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ para un primo p y un punto base $G \in E(\mathbb{F}_p)$ de orden n . Tanto la curva como el punto base y su orden son públicos, y pueden ser conocidos por todos los agentes, incluyendo Eva.
2. Alicia elige un entero $1 < d < n$ de manera aleatoria, el cual será su clave privada.
3. Alicia calcula su clave pública Q utilizando la siguiente fórmula:

$$Q = dG \in E(\mathbb{F}_p)$$

y la comparte con Beto.

3.3.2. Protocolo ECDSA para la generación de firmas digitales

El algoritmo ECDSA para la generación de firmas digitales es el siguiente:

1. Alicia elige un mensaje llano m que desea firmar digitalmente y calcula su hash

$$h = \text{hash}(m)$$

donde hash es una función de hasheo que es criptográficamente segura, como SHA-256.

2. Alicia elige un entero k de manera aleatoria tal que $1 < k < n$ y $\text{mcd}(k, n) = 1$. Es importante que k sea único para cada firma, ya que la reutilización de k puede comprometer la seguridad de la firma.
3. Alicia calcula el punto R utilizando la siguiente fórmula:

$$R = kG \in E(\mathbb{F}_p)$$

y define $r = R_1$, es decir, la primera coordenada del punto R . Si $r = 0$, Alicia debe elegir un nuevo valor de k y repetir este paso.

4. Alicia calcula

$$s = \frac{h + dr}{k} \pmod{n}$$

Si $\text{mcd}(s, n) \neq 1$, Alicia debe elegir un nuevo valor de k y repetir este proceso.

5. El mensaje firmado es

$$\text{mensaje_firmado} = (m, r, s)$$

3.3.3. Protocolo ECDSA para la verificación de firmas digitales

El algoritmo ECDSA para la verificación de firmas digitales es el siguiente:

1. Beto recibe el mensaje firmado (m, r, s) y calcula el hash del mensaje llano m :

$$h = \text{hash}(m)$$

2. Beto calcula

$$w = s^{-1} \pmod{n}$$

$$u_1 = hw \pmod{n}$$

$$u_2 = rw \pmod{n}$$

$$P = u_1G + u_2Q \in E(\mathbb{F}_p)$$

3. Beto verifica que $P \neq \infty$ y define $p = P_1$, es decir, la primera coordenada del punto P . Si $p = r$, entonces la firma es válida, de lo contrario, la firma es inválida.

3.3.4. Ejemplo de aplicación del sistema ECDSA

Ilustraremos ahora el sistema ECDSA con un ejemplo concreto. Alicia y Beto acuerdan utilizar la curva elíptica E definida sobre el campo finito $\mathbb{F}_{23} = \mathbb{Z}/23\mathbb{Z}$ y dada por la ecuación

$$y^2 = x^3 + 3x + 2$$

Notemos que el discriminante de esta curva está dado por

$$\Delta = 17 \pmod{23}$$

lo que nos garantiza que la curva no tiene puntos singulares. El punto base elegido por Alicia y Beto es $G = (5, 2)$, el cual tiene orden $n = 28$. Alicia elige la clave privada $d = 6$ y calcula su clave pública Q utilizando la fórmula dada anteriormente:

$$Q = 6G = 6(5, 2) = (4, 3) \in E(\mathbb{F}_{23})$$

Alicia desea firmar digitalmente el mensaje $m = 15$ y utiliza como función de hash la función identidad, es decir, $\text{hash}(m) = m$. Por lo tanto, el hash del mensaje llano es $h = 15$. Alicia elige un

entero k de manera aleatoria, por ejemplo $k = 9$, y verifica que $\text{mcd}(k, n) = 1$. Luego, Alicia calcula el punto R utilizando la fórmula dada anteriormente:

$$R = 9G = 9(5, 2) = (2, 19) \in E(\mathbb{F}_{23})$$

En particular, $r = R_1 = 2$. Luego, Alicia calcula s utilizando la fórmula dada anteriormente:

$$s = \frac{15 + 6 \cdot 2}{9} = 3 \quad \text{mód } 28$$

Por lo tanto, el mensaje firmado es

$$\text{mensaje_firmado} = (15, 2, 3)$$

Beto recibe el mensaje firmado $(15, 2, 3)$ y calcula el hash del mensaje llano m que en este caso es simplemente $h = 15$. Luego, Beto calcula w , u_1 , u_2 y P utilizando las fórmulas dadas anteriormente:

$$w = 3^{-1} = 19 \quad \text{mód } 28$$

$$u_1 = 15 \cdot 19 = 5 \quad \text{mód } 28$$

$$u_2 = 2 \cdot 19 = 10 \quad \text{mód } 28$$

$$P = 5G + 10Q = 5(5, 2) + 10(4, 3) = (2, 19) \in E(\mathbb{F}_{23})$$

En particular, $p = P_1 = 2$. Como $p = r$, entonces la firma es válida.

3.3.5. Detalles técnicos del sistema ECDSA

Un detalle técnico importante del sistema ECDSA es que el número k utilizado para generar la firma digital debe ser único para cada firma. Si se reutiliza el mismo valor de k para firmar dos mensajes diferentes, entonces, de las fórmulas

$$s_1 = \frac{h_1 + dr}{k} \quad \text{mód } n$$

y

$$s_2 = \frac{h_2 + dr}{k} \quad \text{mód } n$$

obtenemos que

$$k = \frac{h_1 - h_2}{s_1 - s_2} \quad \text{mód } n$$

A partir de este valor de k , un atacante podría calcular la clave privada d utilizando la fórmula

$$d = \frac{s_1 k - h_1}{r} \quad \text{mód } n$$

Observación 3.6. Debemos notar que este fue el error que cometió Sony en 2011 al generar firmas digitales para sus consolas PlayStation 3. Sony utilizó el mismo valor de k para generar varias firmas digitales, lo que permitió a los atacantes calcular la clave privada de Sony y comprometer la seguridad de las consolas PlayStation 3. Este incidente es un ejemplo claro de la importancia de utilizar un valor único de k para cada firma digital generada con el sistema ECDSA.